Modeling Horse Race Outcome

Contents

1 Introduction **2**

2 Horse Racing Context and HKJC Data **2**
2.1 Race Structure and Conditions . . . 2
2.2 HKJC Data Collection and Publication . . . 3
2.3 HKJC Procedures and Data Quality . . . 3

3 Data Representation and Notation **3**
3.1 Multi-Index Structure . . . 3
3.2 Outcome Variables . . . 4

4 Binary Logistic-Loss Boosted Trees **4**
4.1 Logistic Loss and Probability Modeling . . . 4
4.1.1 Gradient and Hessian . . . 5
4.2 Gradient Boosting with Decision Trees . . . 5
4.2.1 Algorithm Outline . . . 5
4.3 Hyperparameters . . . 6
4.4 Probability Calibration via `CalibratedClassifierCV` . . . 6
4.5 Normalizing Probabilities by Race . . . 7
4.6 Independence Assumptions in the Binary Model . . . 7

5 Exploded Logit Model **8**
5.1 Exploded Logit Likelihood . . . 8
5.2 Exploded-Logit Negative Log-Likelihood . . . 8
5.2.1 Gradient and Hessian for One Race . . . 9
5.3 Custom Objective for XGBoost / LightGBM . . . 10
5.4 Prediction: First-Place Probabilities . . . 11

6 Implementation Details **11**
6.1 Preparing Group Pointers . . . 11
6.2 Handling Ties or Missing Positions . . . 11
6.3 Cross-Validation in the Exploded-Logit Model . . . 11

7 Practical Concerns **12**
7.1 Noise in Finishing-Position Labels . . . 12
7.2 Strategic Jockey Behavior . . . 12

8 The Kelly Criterion and Profit Maximization in HKJC **13**
8.1 The Kelly Criterion 13
8.2 Profit Computation After Applying Kelly 13
8.3 Multivariate Kelly Criterion 14
8.4 Bet Simulation / Profit Calculation 15

1 Introduction

This document provides a comprehensive mathematical treatment of two predictive frameworks used for forecasting the probability that a horse will win a race

(a) A *binary classification* approach based on gradient-boosted decision trees (XGBoost), logistic loss, calibration, and post-hoc normalization within each race.

(b) An *exploded logit* approach, which directly models the full ranking of horses in a race using a custom objective function and gradient boosting.

We will derive each model from first principles, covering:

- Data representation and notation.
- The binary logistic-loss boosting algorithm: loss function, gradients, Hessians, tree-based fitting, calibration, and normalization by race.
- The exploded logit model: likelihood, negative log-likelihood, gradients, Hessians, and integration into XGBoost or LightGBM.
- Practical considerations: grouping horses by race, implementation details, and differences between the two approaches.

2 Horse Racing Context and HKJC Data

In Hong Kong—and specifically under the management of the Hong Kong Jockey Club (HKJC)—horse racing is highly regulated and standardized. The following subsections describe the typical conditions of a race, how HKJC organizes and publishes data, and the key variables that feed into our forecasting models.

2.1 Race Structure and Conditions

- **Race Types and Classes.** Each race is classified by its *distance* (e.g. 1,200m, 1,600m, 2,000m), *surface* (turf or all-weather), and *class tier* (Class 1 through Class 5, or Group races). Lower-numbered classes indicate higher-quality fields (e.g. Class 1 is more competitive than Class 5).
- **Track Configuration.** Hong Kong has two main racecourses—Sha Tin and Happy Valley—each with multiple track layouts. A "Straight" 1,000 m course is used for some sprints, while most races use the circular turf courses.

- **Weather and Going.** Weather conditions such as rainfall, humidity, and wind can alter the track surface even within a single race meeting. Prior to each race, the HKJC assigns a "Going" rating that reflects the firmness of the turf. These ratings, in descending order of firmness, are: FAST, GOOD TO FIRM, GOOD, WET FAST, GOOD TO YIELDING, YIELDING, YIELDING TO SOFT, SOFT, WET SLOW, and HEAVY. The going rating is a critical covariate in performance modeling, as certain horses perform better on firmer surfaces, while others are more effective on softer ground.

- **Draw or Barrier Position.** The *draw* (a number from 1 to m_r) determines a horse's stall position at the start. On certain track layouts (especially sprint distances), a high or low barrier draw can confer a tangible advantage or disadvantage.

- **Weight Carried.** Each horse carries a fixed *assigned weight* based on handicap conditions or weight-for-age allowances. This is usually expressed in kilograms (e.g. 52 kg) and affects speed—heavier loads generally slow a horse down.

2.2 HKJC Data Collection and Publication

HKJC publishes a *racecard* for every meeting (race day), typically including:

- **Horse Details:** Name, age, sex (colt, gelding, filly), trainer, jockey, last run performance, win–place–show statistics.

- **Betting Market Data:** Starting prices or *win odds* from on-course tote and off-track bookmakers. *Implied probability* can be extracted as $p_{\text{implied}} = \frac{1}{\text{odds (decimal)}}$. These market odds reflect collective wisdom and often contain information not found in raw feature columns.

Note that some of the variables above are not currently available in the dataset, as HKJC does not publish all the data after the race has concluded.

2.3 HKJC Procedures and Data Quality

- **Verification of Positions:** If a horse is scratched before the race, it never appears in final positions; if it is a "non-runner" on race day, it typically carries a code like "NR." We drop those entries entirely.

- **Handling Dead-Heats or Ties:** Although extremely rare in HK racing, if two horses finish in a *dead-heat* (tie), prize money is split. Our standard approach is to either (a) drop the race from exploded-logit training, or (b) assign each tied horse half-weights in the gradient (e.g. split the "-1" gradient penalty equally).

- **Consistency Checks:** Before any modeling, we verify for each race that finishing positions form a contiguous set $\{1, 2, \ldots, m_r\}$. Races with gaps (DNF, UR) or invalid data are excluded or cleaned.

3 Data Representation and Notation

3.1 Multi-Index Structure

We assume a dataset of horse races. Each observation corresponds to one horse in one race. We index each observation by a *multi-index*:

$$(\text{race_id}, \text{horse_id}).$$

Here:

- race_id $\in \{1, 2, \ldots, R\}$ uniquely identifies each race.
- horse_id $\in \{1, 2, \ldots, m_r\}$ identifies each horse within race r. Each race r has m_r horses.

Let the total number of horses (rows) in the dataset be

$$N \;=\; \sum_{r=1}^{R} m_r.$$

We collect features into a matrix

$$X \in \mathbb{R}^{N \times p},$$

where p is the number of explanatory variables. We denote by

$$x_i \in \mathbb{R}^p$$

the feature-vector for the i-th observation (horse), and assume i is ordered so that horses from the same race appear consecutively whenever convenient (but not required).

3.2 Outcome Variables

(i) **Binary win indicator**:

$$y_i^{\text{win}} = \begin{cases} 1, & \text{if horse } i \text{ wins its race,} \\ 0, & \text{otherwise.} \end{cases}$$

Exactly one horse per race has $y_i^{\text{win}} = 1$.

(ii) **Ordinal finishing position**:

$$y_i^{\text{place}} \in \{1, 2, \ldots, m_r\}$$

is the finishing position of horse i. If horse i in race r finishes in k-th place, then $y_i^{\text{place}} = k$. We assume the data are clean so that exactly one horse has each position $1, \ldots, m_r$ for finishing horses (ties or gaps will be discussed later).

We will develop:

- A *binary-classification* model predicting $\mathbb{P}(y_i^{\text{win}} = 1 \mid x_i)$.
- An *exploded-logit* model predicting $\mathbb{P}(\text{horse } i \text{ finishes first} \mid \{x_j : j \in r\})$ by modeling the entire ranking in race r.

4 Binary Logistic-Loss Boosted Trees

4.1 Logistic Loss and Probability Modeling

We treat each horse as a separate binary-labeled example. Given a vector of features $x_i \in \mathbb{R}^p$, we model the *log-odds* (logit) of winning via a function $F(x_i) \in \mathbb{R}$. The probability of winning is

$$p_i \;=\; \sigma\big(F(x_i)\big) \;=\; \frac{1}{1 + \exp\big(-F(x_i)\big)}.$$

We fit $F(\cdot)$ by minimizing the *binary logistic loss* over the training set:

$$\mathcal{L}_{\text{logistic}}(F) \;=\; \sum_{i=1}^{N}\Big[-\,y_i^{\text{win}}\,\log p_i \;-\; (1-y_i^{\text{win}})\,\log\big(1-p_i\big)\Big].$$

Equivalently, define

$$\ell_i\big(F(x_i)\big) \;=\; \begin{cases} -\log\big(\sigma(F(x_i))\big), & \text{if } y_i = 1,\\ -\log\big(1-\sigma(F(x_i))\big), & \text{if } y_i = 0. \end{cases}$$

Then $\mathcal{L}_{\text{logistic}}(F) = \sum_{i=1}^{N} \ell_i(F(x_i))$.

4.1.1 Gradient and Hessian

To apply gradient boosting, we need first and second derivatives of ℓ_i with respect to $F(x_i)$:

$$\begin{aligned} p_i &= \sigma(F(x_i)),\\ \frac{\partial \ell_i}{\partial F(x_i)} &= p_i - y_i,\\ \frac{\partial^2 \ell_i}{\partial F(x_i)^2} &= p_i\,\big(1-p_i\big). \end{aligned}$$

Thus the *pseudo-residual* is

$$r_i \;=\; -\frac{\partial \ell_i}{\partial F(x_i)} \;=\; y_i - p_i,$$

and the *second-derivative weight* is

$$w_i \;=\; \frac{\partial^2 \ell_i}{\partial F(x_i)^2} \;=\; p_i\,(1-p_i).$$

4.2 Gradient Boosting with Decision Trees

We build $F(x)$ as a sum of T regression trees of maximum depth D:

$$F_T(x) \;=\; \sum_{m=1}^{T} \nu\, h_m(x),$$

where:

- $\nu \in (0,1]$ is the *learning rate* (shrinkage).
- $h_m(x)$ is a regression tree (CART) of depth at most D, fit to the pseudo-residuals at iteration m.

4.2.1 Algorithm Outline

1. Initialize $F_0(x) = 0$ for all x.
2. **For** $m = 1, \ldots, T$:
   a. Compute current probabilities:

$$p_i^{(m-1)} \;=\; \sigma\big(F_{m-1}(x_i)\big).$$

b. Compute pseudo-residuals and weights:

$$r_i^{(m)} \;=\; y_i^{\text{win}} - p_i^{(m-1)}, \quad w_i^{(m)} \;=\; p_i^{(m-1)}\big(1 - p_i^{(m-1)})\big).$$

c. Fit regression tree $h_m(\cdot)$ of depth $\leq D$ to the targets $\{r_i^{(m)}\}$, using weights $\{w_i^{(m)}\}$ (e.g. via weighted squared-error loss).

d. Compute leaf values $\gamma_j^{(m)}$ for each leaf j of h_m, typically:

$$\gamma_j^{(m)} \;=\; \frac{\sum_{i \in \text{leaf}_j} r_i^{(m)}}{\sum_{i \in \text{leaf}_j} w_i^{(m)}}.$$

e. Update:

$$F_m(x) \;=\; F_{m-1}(x) \;+\; \nu\, h_m(x).$$

3. **End For**

After T rounds, final logit is $F_T(x)$ and predicted probability of win is

$$\hat{p}_i \;=\; \sigma\big(F_T(x_i)\big).$$

4.3 Hyperparameters

- $T =$ `n_estimators`: number of trees (boosting rounds).
- $\nu =$ `learning_rate`: shrinkage factor in $(0, 1]$.
- $D =$ `max_depth`: maximum depth of each tree (controls complexity).
- `subsample` $= r$: fraction of training rows sampled (without replacement) for each tree to reduce overfitting.
- `colsample_bytree` $= c$: fraction of features randomly chosen per tree.

4.4 Probability Calibration via `CalibratedClassifierCV`

Raw probabilities $\hat{p}_i = \sigma(F_T(x_i))$ can be mis-calibrated. We use `CalibratedClassifierCV` with K-folds ($K =$ `cv_value`) to improve calibration:

1. Split the training set into K folds of approximately equal size (stratified by year or race if needed). Label these folds as $F_1, \ldots, F_K$.
2. For $k = 1, \ldots, K$:
   - Train an XGBoost model on the union of folds $\bigcup_{\ell \neq k} F_\ell$.
   - Predict raw scores on fold F_k, obtaining $\hat{p}_i^{(k)}$ for $i \in F_k$.
   - Fit a calibration model on $(\hat{p}_i^{(k)}, y_i^{\text{win}})$ for $i \in F_k$.
3. Refit XGBoost on *all* training data to get a final booster $F_T^{\text{full}}(x)$.
4. To calibrate any new (or test) prediction $\hat{p}_i^{\text{raw}} = \sigma\big(F_T^{\text{full}}(x_i)\big)$, apply the learned calibration mapping (e.g. isotonic) that was averaged or selected from the K curves.

In practice, one often uses `CalibratedClassifierCV(XGBClassifier(...)`, $cv = K, \ldots$) which automates this procedure.

4.5 Normalizing Probabilities by Race

Even after calibration, we want each race's horses to form a discrete distribution summing to 1. Let

$$\tilde{p}_i = \text{calibrated probability of win for horse } i.$$

Group by race: for race r, let

$$H_r = \{\, i : \text{horse } i \text{ belongs to race } r \,\}, \quad \widetilde{Z}_r = \sum_{i \in H_r} \tilde{p}_i.$$

Define *normalized* probabilities:

$$P_i \;=\; \frac{\tilde{p}_i}{\widetilde{Z}_r}, \qquad i \in H_r.$$

Then $\sum_{i \in H_r} P_i = 1$ for every race.

4.6 Independence Assumptions in the Binary Model

The binary-classification approach treats each horse as an independent example, fitting a logistic-loss gradient-boosted tree to predict

$$\mathbb{P}(y_i^{\text{win}} = 1 \mid x_i).$$

However, in reality, the outcomes of all horses in the same race are *not* independent:

- If Horse A crosses the finish line first, that *precludes* any other Horse B in the same race from winning (at least, in the case where races are not tied). Thus, learning a model that assumes

  $$\mathbb{P}(y_i^{\text{win}} = 1 \mid x_i) \quad \text{independent of} \quad \{y_j^{\text{win}} : j \neq i,\, \text{race}(j) = \text{race}(i)\}$$

  is an approximation. In truth,

  $$\sum_{i \in H_r} \mathbf{1}\{y_i^{\text{win}} = 1\} \;=\; 1$$

  for each race r, which couples the events within H_r.

- The logistic loss

  $$\ell_i(F(x_i)) = -\big[y_i^{\text{win}} \log \sigma(F(x_i)) + (1 - y_i^{\text{win}}) \log(1 - \sigma(F(x_i)))\big]$$

  is minimized independently for each i. During training, the gradient for horse i is

  $$\frac{\partial \ell_i}{\partial F(x_i)} \;=\; \sigma(F(x_i)) - y_i^{\text{win}},$$

  implicitly assuming y_i^{win} is "unaffected" by other horses' features. In fact, if two horses have very similar covariates x_i and x_j, the model treats them as nearly interchangeable—even though only one can win and the other must lose.

- Because of this independence assumption, the raw $\hat{p}_i = \sigma\big(F_T(x_i)\big)$ across all horses in a given race r generally do *not* sum to 1. We enforce a valid race-level distribution only by a *post-hoc normalization*:

  $$P_i = \frac{\hat{p}_i}{\sum_{j \in H_r} \hat{p}_j}, \quad i \in H_r.$$

  This normalization corrects for dependence only after the fact, rather than embedding it directly in the loss function.

- In essence, the binary model learns an approximation of the true joint distribution

$$\mathbb{P}\big(\,(y_i^{\text{win}})_{i=1}^N\,\big) \;=\; \prod_{i=1}^{N} \mathbb{P}\big(y_i^{\text{win}} \mid x_i\big) \quad \text{(implicitly)},$$

  whereas the actual distribution must satisfy

$$\sum_{i \in H_r} y_i^{\text{win}} = 1, \quad \forall r.$$

5 Exploded Logit Model

5.1 Exploded Logit Likelihood

Suppose a race r has m_r horses. Denote by $\boldsymbol{f}^{(r)} \in \mathbb{R}^{m_r}$ the *score* parameters for those horses. Under a *exploded logit* model, the probability that horse i finishes first in race r is

$$\mathbb{P}(\text{horse } i \text{ is 1st} \mid \boldsymbol{f}^{(r)}) \;=\; \frac{\exp(f_i^{(r)})}{\sum_{j=1}^{m_r} \exp(f_j^{(r)})}.$$

Conditioning on the identity of the first-place finisher i_1, the probability that horse i_2 finishes second (among the remaining $m_r - 1$) is

$$\mathbb{P}(\text{horse } i_2 \text{ is 2nd} \mid i_1, \boldsymbol{f}^{(r)}) \;=\; \frac{\exp(f_{i_2}^{(r)})}{\sum_{j \neq i_1} \exp(f_j^{(r)})}.$$

Continuing in this "exploded" (successive conditional) fashion, the joint probability of the full ranking $(i_1, i_2, \ldots, i_{m_r})$ is

$$\mathbb{P}(i_1, i_2, \ldots, i_{m_r} \mid \boldsymbol{f}^{(r)}) \;=\; \prod_{k=1}^{m_r} \frac{\exp\big(f_{i_k}^{(r)}\big)}{\sum_{j \,\in\, \{i_k, \ldots, i_{m_r}\}} \exp\big(f_j^{(r)}\big)}.$$

This is the exploded logit likelihood. If we only observe the finishing position labels $y_i^{\text{place}} \in \{1, \ldots, m_r\}$ without explicit permutation notation, we can rewrite the negative log-likelihood race-by-race.

5.2 Exploded-Logit Negative Log-Likelihood

For race r, let the horses be indexed $1, \ldots, m_r$ and let $y_{r,i} \in \{1, \ldots, m_r\}$ be the finishing position of horse i in that race (1 is first, 2 is second, ...). Define

$$\pi_{r,i} = y_{r,i} - 1 \quad \text{(zero-based position, } 0 \leq \pi_{r,i} \leq m_r - 1).$$

We want to model scores $f_{r,i} \in \mathbb{R}$ such that higher $f_{r,i}$ corresponds to better finishing. The negative log-likelihood for one race r is:

$$\ell^{(r)}(\{f_{r,i}\}) \;=\; -\log\Big[\,\mathbb{P}(\text{observed ranking} \mid \{f_{r,i}\})\Big].$$

Writing it out explicitly:

$$\ell^{(r)} \;=\; -\sum_{k=0}^{m_r-1} \log\Big[\exp\big(f_{r,i_k}\big) \,/\, \sum_{j:\pi_{r,j}\geq k} \exp\big(f_{r,j}\big)\Big],$$

where i_k is the horse whose zero-based finishing position equals k. Equivalently, indexing directly by horse i, we sum over $k = 0, \ldots, m_r - 1$:

$$\ell^{(r)} = -\sum_{k=0}^{m_r-1} \left[f_{r,i_k} - \log\Big(\sum_{j:\pi_{r,j}\geq k} \exp(f_{r,j}) \Big) \right].$$

Summing across all races,

$$\mathcal{L}_{\mathrm{PL}} = \sum_{r=1}^{R} \ell^{(r)} = -\sum_{r=1}^{R} \sum_{k=0}^{m_r-1} \Big[f_{r,i_k} - \log\big(\sum_{j:\pi_{r,j}\geq k} \exp(f_{r,j}) \big) \Big].$$

5.2.1 Gradient and Hessian for One Race

Fix race r, let $f_r = (f_{r,1}, \ldots, f_{r,m_r})^\top$, and let $\pi_r = (\pi_{r,1}, \ldots, \pi_{r,m_r})$. Define for each stage $k = 0, \ldots, m_r - 1$:

$$A_k(r) \;=\; \{\, i : \pi_{r,i} \geq k \} \quad \text{(horses still “eligible” at stage } k\text{)},$$

and

$$Z_k(r) \;=\; \sum_{j\in A_k(r)} \exp(f_{r,j}), \quad p_{r,j|k} \;=\; \frac{\exp(f_{r,j})}{Z_k(r)}, \quad j \in A_k(r).$$

The partial derivative of $\ell^{(r)}$ with respect to $f_{r,i}$ is:

$$\frac{\partial \ell^{(r)}}{\partial f_{r,i}} = -\sum_{k=0}^{m_r-1} \left[\frac{\partial}{\partial f_{r,i}} f_{r,i_k}\, \mathbb{I}\{i = i_k\} \;-\; \frac{\partial}{\partial f_{r,i}} \log Z_k(r) \right].$$

But

$$\frac{\partial}{\partial f_{r,i}} f_{r,i_k}\, \mathbb{I}\{i = i_k\} = \mathbb{I}\{\pi_{r,i} = k\}, \quad \frac{\partial}{\partial f_{r,i}} \log Z_k(r) = \frac{\exp(f_{r,i})}{Z_k(r)}\, \mathbb{I}\{i \in A_k(r)\} = p_{r,i|k}\, \mathbb{I}\{\pi_{r,i} \geq k\}.$$

Hence

$$\frac{\partial \ell^{(r)}}{\partial f_{r,i}} = -\sum_{k=0}^{m_r-1} \Big[\mathbb{I}\{\pi_{r,i} = k\} \;-\; p_{r,i|k}\, \mathbb{I}\{\pi_{r,i} \geq k\} \Big].$$

Grouping terms:

$$\frac{\partial \ell^{(r)}}{\partial f_{r,i}} = -\sum_{k\,:\,k\leq \pi_{r,i}} \Big[-p_{r,i|k} \Big] \;-\; \mathbb{I}\{\pi_{r,i} = k\}\Big|_{k=\pi_{r,i}} = -\big[\,(1) \underbrace{1}_{\mathbb{I}\{\pi_{r,i}=\pi_{r,i}\}} - p_{r,i|\pi_{r,i}}\big] - \sum_{k<\pi_{r,i}} \big[-p_{r,i|k}\big].$$

A more compact way: for each i,

$$\frac{\partial \ell^{(r)}}{\partial f_{r,i}} = -\sum_{k=0}^{\pi_{r,i}-1} \big(-p_{r,i|k}\big) \;-\; \Big(1 - p_{r,i|\pi_{r,i}}\Big) = \sum_{k=0}^{\pi_{r,i}-1} p_{r,i|k} \;+\; p_{r,i|\pi_{r,i}} \;-\; 1.$$

Alternatively,

$$\frac{\partial \ell^{(r)}}{\partial f_{r,i}} = -\sum_{k\,:\,\pi_{r,i}=k} \underbrace{1}_{\mathbb{I}} + \sum_{k=0}^{m_r-1} p_{r,i|k}\,\mathbb{I}\{\pi_{r,i} \geq k\}.$$

The *Hessian* (second derivative) is

$$\frac{\partial^2 \ell^{(r)}}{\partial f_{r,i}^2} = \sum_{k=0}^{\pi_{r,i}} p_{r,i|k}\big(1 - p_{r,i|k}\big).$$

All cross-derivatives $\partial^2 \ell^{(r)} / \partial f_{r,i}\, \partial f_{r,j}$ for $i \neq j$ are zero (the objective decouples across horses within a single stage k).

5.3 Custom Objective for XGBoost / LightGBM

To embed into XGBoost's 'xgb.train' or LightGBM's 'lgb.train', we define:

- **Predictions vector** $\boldsymbol{f} \in \mathbb{R}^N$ stacking all races' scores in the same order as X.
- **Labels vector** $\boldsymbol{\pi} \in \{0, 1, \dots\}^N$ giving zero-based positions for each horse.
- **Group pointer** array `group_ptr` of length $R + 1$ such that

$$\texttt{group_ptr}[0] = 0, \quad \texttt{group_ptr}[r] = \sum_{t=1}^{r} m_t, \quad \texttt{group_ptr}[R] = N.$$

- For each race r, the index range $[s, e) = [\texttt{group_ptr}[r-1], \texttt{group_ptr}[r])$ corresponds to the m_r horses of that race.

We implement a function

$$(\boldsymbol{g}, \boldsymbol{h}) = \texttt{exploded_logit_obj}(\boldsymbol{f}, \texttt{data})$$

where:

$$g_i = \frac{\partial \mathcal{L}_{\mathrm{PL}}}{\partial f_i}, \qquad h_i = \frac{\partial^2 \mathcal{L}_{\mathrm{PL}}}{\partial f_i^2},$$

computed race-by-race using the above formulas. Then in XGBoost:

```
bst = xgb.train(
    params,
    DMatrix(X, label=pi),
    num_boost_round = T,
    obj = exploded_logit_obj,
    group = group_ptr
)
```

5.4 Prediction: First-Place Probabilities

Once the booster $\{h_m(\cdot)\}_{m=1}^{T}$ is trained, we obtain scores

$$f_i^\star = F_T(x_i)$$

for each horse in the test set. To get the probability that horse i in race r finishes first, we apply a *softmax* across those m_r horses:

$$p_i^{(1\text{st})} \;=\; \frac{\exp(f_i^\star)}{\sum_{j\in H_r}\exp(f_j^\star)}, \quad i \in H_r.$$

We do not explicitly model 2nd, 3rd, etc., if we only care about first-place: those intermediate stages are used in training but at prediction time we require just the top-slice softmax.

6 Implementation Details

6.1 Preparing Group Pointers

Given the training set X_{train} indexed by (race_id, horse_id), we compute:

$$\texttt{grp_sizes} = \big(m_1, m_2, \ldots, m_{R_{\text{train}}}\big), \quad \texttt{group_ptr} = \Big[\,0,\, m_1,\, m_1+m_2,\, \ldots,\, \sum_{r=1}^{R_{\text{train}}} m_r\,\Big].$$

In Python:

```
def prepare_group_sizes(X):
    races = X.index.get_level_values(0)
    return races.value_counts(sort=False)\
                .loc[races.unique()]\
                .values
```

6.2 Handling Ties or Missing Positions

If a race has tied finishing positions or gaps (e.g. horses DNF), the indices $\{0, \ldots, m_r - 1\}$ may not align with actual $\pi_{r,i}$. Before training, one can:

- **Drop** any race that does not have positions $\{1, 2, \ldots, m_r\}$ exactly once each.
- **Modify** the gradient logic to split the (-1) penalty across all horses tied for a given stage (e.g. if two horses tie for 1st, each gets $-1/2$).

In practice, dropping these rare tied races is simplest, as these occurences are relatively rare. Notice that this only occurs in "training data".

6.3 Cross-Validation in the Exploded-Logit Model

In the binary-classification pipeline, we employ `CalibratedClassifierCV` with a `cv_value` parameter to perform K-fold calibration on the training set. By splitting training data into K folds (stratified by race or season), we learn an isotonic mapping from raw probabilities to calibrated probabilities, then retrain on all data.
By contrast, the exploded-logit implementation does *not* include a built-in `cv_value` parameter inside its custom objective. Concretely:

- XGBoost's `xgb.train` (and LightGBM's `lgb.train`) do not accept a single flag that automatically performs cross-validation *and* calibration for a custom objective.
- To obtain similarly calibrated probabilities under an exploded-logit loss, we must manually conduct a Group-K-Fold loop outside of `xgb.train`. In each fold:
  1. Train the exploded-logit booster on $K-1$ folds (groups of races).
  2. Predict raw first-place scores on the held-out fold.
  3. Apply per-race softmax on those scores.
  4. Fit an `IsotonicRegression` on {softmaxed probabilities, $\mathbf{1}\{y_i^{\text{place}} = 1\}$} for the held-out fold.
  5. Store out-of-fold calibrated probabilities.
- After cycling through all folds, we retrain the final exploded-logit booster on *all* training races and apply the last fold's isotonic mapping to test-set probabilities.
- This manual procedure implicitly replicates what `CalibratedClassifierCV` does for binary logistic boosting, but it requires roughly `cv_value` separate fits of the exploded-logit model.
- In effect, `cv_folds` in the exploded-logit function is a "wrapper" that triggers this external Group-K-Fold + calibration loop; no cross-validation is embedded within the objective itself.

Thus, although we can achieve similar calibration for exploded-logit probabilities, it is not a one-line flag inside XGBoost. Instead, it requires explicit fold management in Python, which can be computationally expensive when `cv_folds` > 1.

7 Practical Concerns

Even the exploded-logit model, which directly optimizes a exploded logit likelihood over full race orderings, faces real-world complications:

7.1 Noise in Finishing-Position Labels

- **Non-Finishers, Disqualifications, and Dead-Heats.** Occasionally a horse is pulled up (PU) or does not finish (DNF) due to injury. If multiple horses finish in a dead-heat, the "tie" may be recorded as equal positions or as an averaged payout. Standard exploded-logit derivations assume an unambiguous total order, so any ties or DNFs must be either excluded or specially handled (e.g. split gradient contributions). These occurences are relatively rare.
- **Varying Field Size.** Race sizes m_r vary from as few as 5 horses to as many as 14 (or more). Noisy labels in small fields can disproportionately affect gradient estimates, especially early in boosting when only a few trees have been fit.

7.2 Strategic Jockey Behavior

- **Conditional Effort.** In a handicapped system like HKJC's, jockeys may non-optimally "settle" a horse if they sense it cannot win. Instead of pushing for 2nd or 3rd, they might ease the mount to conserve the horse for future races, especially if the likely finishing position offers little prize money. This introduces behavioral noise: a horse that is objectively "2nd-best" by raw speed may intentionally underperform to save energy.

- **Tactical Riding.** Depending on track bias (e.g. inside rail favoring speed early on) or anticipated pace scenario, jockeys might slow a strong horse early, sacrificing a high finish in one race to preserve serviceability across multiple races in a week. Such tactics violate the assumption that y_i^{place} is generated purely by latent fitness parameters.

- **Effect on Exploded-Logit Loss.** Because Plackett–Luce assumes each horse's probability of finishing k-th depends monotonically on $f_{r,i}$, any intentional underperformance skews the gradient. The model may over-penalize a horse that is fit enough to win but is held back, and conversely under-penalize a horse that closes late only because early leaders deliberately slowed.

8 The Kelly Criterion and Profit Maximization in HKJC

8.1 The Kelly Criterion

Suppose we have a single betting opportunity with odds b (decimal odds minus one, so if the bookmaker pays back $1+b$ on a win) and a probability of winning p. Let the probability of losing be $q = 1-p$. Denote by f the fraction of one's current bankroll W to wager on this single bet. Then:

If you win: $W_{\text{new}} = W \;+\; f\,W\,b = W\,\big(1+f\,b\big)$. If you lose: $W_{\text{new}} = W \;-\; f\,W = W\,\big(1-f\big)$.

The Kelly criterion chooses f to maximize the expected logarithmic growth of wealth:

$$\max_{0\le f\le 1}\Big[\,p\,\log\!\big(1+f\,b\big) \;+\; q\,\log\!\big(1-f\big)\Big].$$

Taking the derivative with respect to f and setting to zero gives the optimal fraction

$$f^\star \;=\; \frac{p\,b \;-\; q}{b} \;=\; \frac{p\,b \;-\; (1-p)}{b} \;=\; \frac{p\,b-(1-p)}{b} \;=\; p \;-\; \frac{q}{b}.$$

Equivalently,

$$f^\star = \frac{p\,b-(1-p)}{b}.$$

8.2 Profit Computation After Applying Kelly

Let W_0 be one's initial bankroll. If one wagers the Kelly-optimal fraction $f^\star$ on a single event with odds b and true win probability p, then:

- **With probability** p (win):
$$W_{\text{win}} = W_0\,\big(1+f^\star\,b\big).$$

- **With probability** $q = 1-p$ (loss):
$$W_{\text{lose}} = W_0\,\big(1-f^\star\big).$$

Because $f^\star$ maximizes the expected logarithmic growth, the long-run (geometric) growth rate is

$$G = p\,\log\!\big(1+f^\star\,b\big) \;+\; q\,\log\!\big(1-f^\star\big).$$

Hence, after n independent, identically distributed repetitions, the expected bankroll on a logarithmic scale is:

$$\mathbb{E}\big[\log W_n\big] = \log W_0 \;+\; n\,G, \quad \Longrightarrow \quad W_n \approx W_0 \,\exp\big(n\,G\big).$$

In practice, one is usually more concerned with the long-term *growth rate* G, because Kelly maximizes $\mathbb{E}[\log W]$, not $\mathbb{E}[W]$. Nevertheless, the arithmetic expectation above highlights that a positive expected profit requires $p\,b - q > 0$, i.e. "the edge" must be positive.

8.3 Multivariate Kelly Criterion

Consider a single race with M mutually exclusive horses. For horse i:

$$p_i = \text{(predicted win probability)}, \quad \text{decimal odds} = o_i, \quad b_i = o_i - 1 \quad \text{(net odds)}.$$

We choose a fraction $f_i \geq 0$ of our bankroll W to bet on horse i. Let

$$F = \sum_{i=1}^{M} f_i, \quad f_0 = 1 - F \text{ (cash remainder)}.$$

If horse j wins, the final bankroll multiplier is

$$R_j = f_0 \;+\; f_j\,(1 + b_j) = \big(1 - F\big) \;+\; f_j\,(1 + b_j).$$

Hence the expected log-growth objective is

$$\sum_{j=1}^{M} p_j \,\log\big(R_j\big) \;=\; \sum_{j=1}^{M} p_j \,\log\Big((1 - F) + f_j\,(1 + b_j)\Big).$$

Equivalently, define

$$R_i(f) = 1 - \sum_{k=1}^{M} f_k \;+\; f_i\,(1 + b_i),$$

and solve

$$\max_{\substack{f_i \geq 0 \\ \sum_{i=1}^{M} f_i \leq 1}} \sum_{i=1}^{M} p_i \,\log\big(R_i(f)\big).$$

This is implemented by minimizing the negative:

$$\texttt{objective}(f) = -\sum_{i=1}^{M} p_i \,\log\big(R_i(f)\big), \quad R_i(f) = 1 - \sum_{k=1}^{M} f_k + (1 + b_i)\,f_i,$$

subject to

$$f_i \geq 0, \quad \sum_{i=1}^{M} f_i \leq 1.$$

8.4 Bet Simulation / Profit Calculation

Once $\{f_i\}$ are computed, let

$$F = \sum_{i=1}^{M} f_i, \quad f_0 = 1 - F.$$

If horse k wins, the race's bankroll multiplier is

$$R_k = f_0 \; + \; f_k \, (1 + b_k).$$

Thus the net profit (return) for that race is

$$\text{race_return} = R_k \; - \; 1.$$

In code, for each race:

- Compute $F = \sum_i f_i$.
- Identify the winning horse(s).
- If exactly one winner k, set

$$\text{final_multiplier} = (1 - F) \; + \; f_k \, (1 + b_k), \quad \text{race_return} = \text{final_multiplier} - 1.$$

- If multiple winners (i.e., a dead heat among n horses), compute each winner's reduced payout:

$$\text{adjusted_payout}_i = \frac{f_i \cdot (1 + b_i)}{n}, \quad i \in \text{winners}.$$

  Then compute

$$\text{final_multiplier} = (1 - F) + \sum_{i \in \text{winners}} \text{adjusted_payout}_i, \quad \text{race_return} = \text{final_multiplier} - 1.$$